

Depth-First Search (DFS)

```
const int N = 1e5 + 5;
vector<int> adj[N];
vector<bool> visited(N);

void dfs(int node) {
    visited[node] = true;
    for (int neighbor : adj[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}
```

Breadth-First Search (BFS)

```
const int N = 1e5 + 5;
vector<int> adj[N];
vector<bool> visited(N);

void bfs(int start) {
    queue<int> q;
    q.push(start);
    visited[start] = true;

    while (!q.empty()) {
        int node = q.front();
        q.pop();
        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

Dijkstra's Algorithm (Shortest Path from 1 source)

```

const int INF = 1e9;
const int N = 1e5 + 5;
vector<pair<int, int>> adj[N]; // {neighbor, weight}
vector<int> dist(N, INF);

void dijkstra(int start) {
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    dist[start] = 0;
    pq.push({0, start}); // {distance, node}

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}

```

Bellman-Ford Algorithm – $O(n \times m)$

```

const int INF = 1e9;
struct Edge {
    int u, v, w;
};

int main() {
    int n, m, src;
    vector<Edge> edges(m);

    vector<int> dist(n + 1, INF);
    dist[src] = 0;

    // Relax all edges n - 1 times
    for (int i = 1; i < n; ++i) {
        for (auto [u, v, w] : edges) {
            if (dist[u] != INF && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }
    // Check for negative weight cycle
    bool negativeCycle = false;

```

```

    for (auto [u, v, w] : edges) {
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            negativeCycle = true;
            break;
        }
    }
}

```

Floyd-Warshall Algorithm – $O(n^3)$

```

const int INF = 1e9;
const int N = 505;
int dist[N][N];

int main() {
    int n, m;
    // Initialize distances
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            dist[i][j] = (i == j ? 0 : INF);

    // Read edges
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        dist[u][v] = min(dist[u][v], w); // in case of multiple edges
    }

    // Floyd-Warshall
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (dist[i][k] < INF && dist[k][j] < INF)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
}

```

Update Bellman-Ford After Adding One Edge $O(n)$:

```

void updateBellmanFord(int u, int v, int w, vector<int>& dist, const
vector<vector<pair<int, int>>>& adj) {
    if (dist[u] + w >= dist[v]) return; // No update needed

    dist[v] = dist[u] + w;
}

```

```

queue<int> q;
q.push(v);

while (!q.empty()) {
    int node = q.front(); q.pop();
    for (auto [nbr, cost] : adj[node]) {
        if (dist[node] + cost < dist[nbr]) {
            dist[nbr] = dist[node] + cost;
            q.push(nbr);
        }
    }
}

```

Update Floyd-Warshall After Adding Edge (u, v, w) $O(n^2)$:

```

void updateFloyd(int u, int v, int w, vector<vector<int>>& dist, int n) {
    if (dist[u][v] <= w) return; // No update needed

    dist[u][v] = w;

    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            if (dist[i][u] < 1e9 && dist[v][j] < 1e9)
                dist[i][j] = min(dist[i][j], dist[i][u] + w + dist[v][j]);
}

```

LCA

```

vector<int>dis(n + 1);
vector<vector<int>>anc(20, vector<int>(n + 1)) ;
function<void(int,int)>dfs=[&](int u, int p ){
    anc[0][u] = p ;
    for(auto f : adj[u]){
        if(f!=p){
            dis[f] = dis[u]+1;
            dfs(f,u);
        }
    }
};

```

```

for (int mask = 1; mask < 18; mask++)
{
    for (int u = 1; u <= n; u++)
    {
        anc[mask][u] = anc[mask-1][anc[mask-1][u]];
        // for max edge in path
        mx [mask][u] = max(mx[mask-1][u] , mx[mask-1][anc[mask-1][u]]) ;
    }
}

auto get_kth_ancestor=[&](int u, int k) {
    for (int i = 0; i < 20; ++i) {
        if (k & (1 << i)) {
            u = anc[i][u];
            if (u == 0) break;
        }
    }
    return u;
};

int getLCA(int u, int v)
{
    if (dis[v] > dis[u]) swap(u, v);

    int diff = dis[u] - dis[v];
    for (int i = 0; diff > 0; ++i) {
        if (diff & 1)
            u = anc[i][u];

        diff >>= 1;
    }

    if (u == v) return u;

    for (int i = anc.size() - 1; i >= 0; --i) {
        if (anc[i][u] != anc[i][v]) {
            u = anc[i][u];
            v = anc[i][v];
        }
    }
    return anc[0][u];
}

```

Flat Tree:

```

int timer = 0 ;
vector<int>newVals(n) ;
function<void(int,int)>dfs = [&](int u , int p){

```

```

    in[u] = timer++;
    for(auto f : adj[u]){
        if(f!=p)dfs(f,u);
    }
    out[u] = timer-1;
};
dfs(1,0);
for (int i = 1; i <= n; i++)
{
    newVals[in[i]] = arr[i-1];
}

```

Flat Tree For Mo Path:

```

function<void(int,int)>dfs=[&](int u , int p){
    anc[0][u] = p ;
    in[u] = timer++;
    for(auto f : adj[u]){
        if(f!=p){
            dis[f] = dis[u]+1;
            dfs(f,u);
        }
    }
    out[u] = timer++;
};
vi vals(n*2) , nodes(n*2);
for (int i = 1; i <= n; i++)
{
    vals[in[i]] = arr[i-1] ;
    vals[out[i]] = arr[i-1];
    nodes[in[i]] = i ;
    nodes[out[i]] = i ;
}

```

DSU:

```

struct Dsu
{
    vector<int> par , sz ;
    Dsu (int n){
        for (int i = 0; i <= n; i++)
        {
            sz.push_back(1);
            par.push_back(i);
        }
    }
}

```

```

    }
}
int find (int u){
    return (par[u] == u ? u : par[u] = find(par[u]));
}

bool merge (int u , int v){
    u = find(u) , v = find(v) ;

    if(u==v)return false ;

    if(sz[v] > sz[u])swap(u,v);
    par[v] = u ;
    sz[u]+=sz[v];
    return true ;
}
};

```

DSU With Roll Back:

```

struct DSU {
    vector<int> par, sz, checkPoints;
    vector<pair<int, int>> updates;

    void init(int n) {
        par.assign(n + 1, {});
        iota(par.begin(), par.end(), 0);
        sz.assign(n + 1, 1);
        checkPoints.clear();
        updates.clear();
    }

    int find(int u) {
        while (u ^ par[u]) {
            u = par[u];
        }
        return u;
    }

    bool merge(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) {
            return false;
        }
        if (sz[u] < sz[v]) {
            swap(u, v);
        }
        updates.emplace_back(v, u);
        sz[u] += sz[v];
    }
};

```

```

    par[v] = u;
    return true;
}

void snapshot() {
    checkPoints.emplace_back(updates.size());
}

void rollback() {
    if (checkPoints.empty()) {
        return;
    }
    while (checkPoints.back() != updates.size()) {
        auto [v, u] = updates.back();
        updates.pop_back();
        sz[u] -= sz[v];
        par[v] = v;
    }
    checkPoints.pop_back();
}

int size(int u) {
    return sz[find(u)];
}

int sameSet(int u, int v) {
    return find(u) == find(v);
}

};

```

MO on Path On Tree :

```

void solve(){
    int n , q , timer = 0 , id = 1; cin>>n>>q ;
    vvi adj(n+1) , anc(20,vi(n+1));
    vi dis (n+1) , in(n+1) , out(n+1) , cnt (n+5) , res(q) , vals(n*2) , nodes(n*2);
    vector<bool> vis(n+1);
    map<int,int>mp;
    int arr[n] ;

    for (int i = 0; i < n; i++)
    {
        cin>>arr[i] ;
        if(mp[arr[i]]==0){
            mp[arr[i]] = id++ ;
        }
        arr[i] = mp[arr[i]] ;
    }
    for (int i = 0; i < n-1; i++)

```



```

{
    int a , b ; cin>>a>>b;
    adj[a].push_back(b);
    adj[b].push_back(a);
}
function<void(int,int)>dfs=[&](int u , int p){
    anc[0][u] = p ;
    in[u] = timer++;
    for(auto f : adj[u]){
        if(f!=p){
            dis[f] = dis[u]+1;
            dfs(f,u);
        }
    }
    out[u] = timer++;
};
dfs(1,0);
vector<array<int,4>>qi ;
for (int i = 0; i < q; i++)
{
    int u , v ; cin>>u>>v;
    if(in[u]>in[v])swap(u,v) ; // Let  $ST(u) \leq ST(v)$ 
    int lc = getLCA(u,v) ;
    if(lc == u){
        qi.push_back({in[u],in[v],i,0}) ; // [ST(u), ST(v)]
    }else{
        qi.push_back({out[u],in[v],i,getLCA(u,v)}) ; // [EN(u), ST(v)] + [ST(P),
ST(P)].
    }
}
int BS = sqrt(2*n)+1;
sort(all(qi),[&](array<int,4>q1,array<int,4>q2){
    return make_pair(q1[0]/BS , q1[1]) < make_pair(q2[0]/BS , q2[1]);
});
for (int i = 1; i <= n; i++)
{
    vals[in[i]] = arr[i-1] ;
    vals[out[i]] = arr[i-1];
    nodes[in[i]] = i ;
    nodes[out[i]] = i ;
}
int ans = 0 ;

auto add = [&](int ind){
    int val = vals[ind] ;
    if(!cnt[val])ans++;
    cnt[val]++;
};
auto ers = [&](int ind){
    int val = vals[ind] ;
    cnt[val]--;
    if(!cnt[val])ans--;
};
auto go = [&](int ind){

```

```

        int u = nodes[ind];
        vis[u] = vis[u]^1 ;
        if(vis[u])add(ind);
        else ers(ind);
    };
    int l = qi[0][0];
    int r = 1 ;
    go(l) ;
    for (auto [lx , rx , ix , lc] : qi)
    {
        while (lx<l)go(--l);
        while (rx>r)go(++r);

        while (lx>l)go(l++);
        while (rx<r)go(r--);

        if(lc)add(in[lc]) ;

        res[ix] = ans ;

        if(lc)ers(in[lc]) ;

    }
    for (int i = 0; i < q; i++)cout<<res[i] ln ;
}

```

BFS 01 :

```

void bfs01(int src) {
    const int n = adj.size();
    const int INF = 1e9;
    vector<int> d(n, INF);
    d[src] = 0;
    deque<int> q;
    q.push_front(src);
    while (!q.empty()) {
        int v = q.front();
        q.pop_front();
        for (auto edge: adj[v]) {
            int u = edge.first;
            int w = edge.second;
            if (d[v] + w < d[u]) {
                d[u] = d[v] + w;
                if (w == 1)
                    q.push_back(u);
                else
                    q.push_front(u);
            }
        }
    }
}

```

```

    }
  }
}

```

LCA in $O(1)$:

```

vector<vector<int>> adj;
vector<int> in, lvl;
vector<vector<int>> euler;

void dfs0(int u, int par) {
    in[u] = euler[0].size();
    euler[0].emplace_back(u);
    for (auto &v: adj[u]) {
        if (par == v) continue;
        lvl[v] = lvl[u] + 1;
        dfs0(v, u);
        euler[0].emplace_back(u);
    }
}

int mrg(int u, int v) {
    return lvl[u] < lvl[v] ? u : v;
}

void precompute_lca(int root = 0) {
    euler = {{}};
    in.assign(adj.size(), {});
    lvl.assign(adj.size(), {});
    dfs0(root, 0);
    const int sz = euler[0].size();
    for (int msk = 1; (1 << msk) <= sz; ++msk) {
        euler.emplace_back(sz - (1 << msk) + 1);
        for (int i = 0; i + (1 << msk) <= sz; ++i) {
            euler[msk][i] = mrg(
                euler[msk - 1][i],
                euler[msk - 1][i + (1 << (msk - 1))]
            );
        }
    }
}

```

Small to Large Tree :

```

const int N = 3e5 + 5;
vector<vector<int>> adj;
int in[N], sz[N], tree[N];
int t; /// reset this to 0 each testcase

void dfs0(int u, int p = -1) {
    tree[t] = u, in[u] = t++;
    sz[u] = 1;
    for (int v: adj[u]) {
        if (v == p) continue;
        dfs0(v, u);
        sz[u] += sz[v];
    }
}

void add(int node) {}
void rem(int node) {}

void dfs1(int u, int p, bool keep) {
    int bg = -1, mx = -1;
    for (int &v: adj[u]) {
        if (v != p && sz[v] > mx)
            mx = v, bg = v;
    }
    for (int &v: adj[u]) {
        if (v == p or v == bg) continue;
        dfs1(v, u, false);
    }
    if (~bg) {
        dfs1(bg, u, true);
    }
    add(u);
    for (int v: adj[u]) {
        if (v == p or v == bg) continue;
        for (int i = 0; i < sz[v]; ++i) {
            int x = tree[in[v] + i];
            add(x);
        }
    }
}

///// get the answers here

if (!keep) {
    for (int i = 0; i < sz[u]; ++i) {
        int x = tree[in[u] + i];
        rem(x);
    }
}
}

```

HLD On Tree :

```

struct HLD
{
    vi big , head , siz , par , id , dpth;
    int timer ;
    HLD(int n , vvi&adj): big(n+1) ,par(n+1), id(n+1), head(n+1), dpth(n+1),
    siz(n+1,1){
        dfs(1 , adj);
        timer = 0 ;
        head[1] = 1 ;
        dfs1(1 , adj);
    }
    void dfs (int u , vvi& adj){
        for(auto f : adj[u]){
            if(f == par[u])continue;
            par[f] = u ;
            dpth[f] = dpth[u]+1 ;
            dfs(f,adj);
            siz[u]+=siz[f] ;
            if(!big[u] || siz[f] > siz[big[u]]) big[u] = f ;
        }
    }
    void dfs1 (int u , vvi& adj){
        id[u] = timer++;
        if(big[u]) head[big[u]] = head[u] ,dfs1(big[u] , adj) ;
        for(auto f : adj[u]){
            if(f == par[u] || f == big[u])continue;
            head[f] = f ;
            dfs1(f , adj);
        }
    }
    vector<pair<int,int>>path(int u , int v){
        vector<pair<int,int>>res ;
        while (true)
        {
            if(head[u] == head[v]){
                if(dpth[u] > dpth[v])swap(u,v);
                res.push_back({id[u] , id[v]});
                return res ;
            }
            if(dpth[head[u]] > dpth[head[v]])swap(u,v);
            res.push_back({id[head[v]],id[v]});
            v = par[head[v]];
        }
    }
}

```

```
};
```

HLD sorted Path :

```
vector<pair<int,int>> path(int u , int v){
    vector<pair<int,int>> left, right;

    while(head[u] != head[v]) {
        if(dpth[head[u]] > dpth[head[v]]) {
            // segment لحد u من head[u]
            left.push_back({id[head[u]], id[u]});
            u = par[head[u]];
        } else {
            // segment لحد v من head[v]
            right.push_back({id[head[v]], id[v]});
            v = par[head[v]];
        }
    }

    // في نفس السلسلة v و u دلوقتى
    if(dpth[u] > dpth[v]) swap(u,v);
    // segment الأخير من u لحد v
    left.push_back({id[u], id[v]});

    // u → (LCA) → v علشان يطلع من left نقلب
    reverse(left.begin(), left.end());

    // ندمج
    left.insert(left.end(), right.begin(), right.end());
    return left;
}
```

Sack in tree

```
auto collect = [&](int u , int d){
    if(freq[arr[u]]==0)ans++;
    freq[arr[u]]+=d;
    if(freq[arr[u]]==0)ans--;
};

function<void(int,int,int)> addSubtree = [&](int u, int p, int val){
    collect(u, val);
    for(auto f : adj[u]){
        if(f == p) continue;
    }
}
```

```

        addSubtree(f, u, val);
    }
};
function<void(int,int)>dfs0=[&](int u ,int p){
    siz[u] = 1 ;
    for(auto f : adj[u]){
        if(f == p)continue;
        dfs0(f , u);
        siz[u] += siz[f];
        if(!big[u] || siz[big[u]] < siz[f])big[u] = f ;
    }
};
function<void(int,int,bool)>dfs1=[&](int u ,int p , bool keep){
    for(auto f : adj[u]){
        if(f == p || f == big[u])continue;
        dfs1(f , u , false);
    }
    if(big[u]) dfs1(big[u] , u , true);

    collect(u , 1);
    for(auto f : adj[u]){
        if(f == p || f == big[u])continue;
        addSubtree(f , u, 1);
    }
    res[u] = ans ;
    if(!keep)addSubtree(u , p, -1);
};
dfs0(1,1);
dfs1(1,1,true);

```

Dinic's Algorithm — $O(V^2 \times E)$

```

struct Edge
{
    int to , rev , cap ;
};
struct Dinic
{
    vector<vector<Edge>>graph;
    vector<int> level, iter;

    int n;

    Dinic(int n) : n(n), graph(n), level(n), iter(n) {}

    void add_e(int u , int v , int cap){
        graph[u].push_back({v , (int)graph[v].size() , cap});
        graph[v].push_back({u , (int)graph[u].size()-1 , 0});
    }
}

```

```

bool bfs(int s , int t){
    fill(level.begin() , level.end() , -1);
    queue<int>q;
    q.push(s);
    level[s] = 0;
    while (!q.empty())
    {
        int u = q.front(); q.pop();
        for(auto [to , rev , cap] : graph[u]){
            if(level[to] == -1 && cap){
                level[to] = level[u] + 1 ;
                q.push(to);
            }
        }
    }
    return level[t] != -1 ;
}

int dfs (int u , int t , int mn){
    if(u == t)return mn ;
    while(iter[u] != graph[u].size()){

        auto &[to , rev , cap] = graph[u][iter[u]];

        if(cap > 0 && level[to] == level[u] +1 ){
            int cur = dfs(to , t , min(mn , cap));
            if(cur){
                cap -= cur ;
                graph[to][rev].cap += cur ;
                return cur ;
            }
        }
        iter[u]++;
    }
    return 0 ;
}

int max_flow(int s , int t){
    int res = 0 ;
    while (bfs(s , t))
    {
        fill(iter.begin(), iter.end(), 0);
        while (true)
        {
            int d = dfs(s, t, 1e18);
            res += d ;
            if(!d) break;
        }
    }
    return res ;
}
};

```

// Usage:


```
// Dinic dinic(n);
// dinic.add_edge(u, v, cap);
// cout << dinic.max_flow(source, sink);
```

Edmonds-Karp (BFS Augmentation) — $O(V \times E^2)$

```
struct EdmondsKarp {
    struct Edge {
        int to, rev;
        long long cap;
    };

    int n;
    vector<vector<Edge>> graph;

    EdmondsKarp(int n) : n(n), graph(n) {}

    void add_edge(int from, int to, long long cap) {
        graph[from].push_back({to, (int)graph[to].size(), cap});
        graph[to].push_back({from, (int)graph[from].size() - 1, 0}); // reverse
    }

    long long max_flow(int s, int t) {
        return bfs(s, t, vector<int>(), vector<int>());
    }

    long long bfs(int s, int t, vector<int>& parent, vector<int>& parent_edge) {
        fill(parent.begin(), parent.end(), -1);
        parent[s] = s;
        queue<pair<int, long long>> q;
        q.push({s, LLONG_MAX});

        while (!q.empty()) {
            int v = q.front().first;
            long long flow = q.front().second;
            q.pop();

            for (int i = 0; i < graph[v].size(); i++) {
                Edge& e = graph[v][i];
                if (parent[e.to] == -1 && e.cap > 0) {
                    parent[e.to] = v;
                    parent_edge[e.to] = i;
                    long long new_flow = min(flow, e.cap);
                    if (e.to == t) return new_flow; // reached sink
                    q.push({e.to, new_flow});
                }
            }
        }
        return 0;
    }

    long long max_flow(int s, int t) {
        long long flow = 0;
        while (long long f = bfs(s, t, parent, parent_edge)) {
            flow += f;
        }
        return flow;
    }
};
```

```

vector<int> parent(n), parent_edge(n);

long long new_flow;
while ((new_flow = bfs(s, t, parent, parent_edge)) > 0) {
    flow += new_flow;
    // Augment along the path
    int v = t;
    while (v != s) {
        int u = parent[v];
        int i = parent_edge[v];
        graph[u][i].cap -= new_flow;
        graph[v][graph[u][i].rev].cap += new_flow;
        v = u;
    }
}
return flow;
}
};

// Usage:
// EdmondsKarp ek(n);
// ek.add_edge(u, v, cap);
// cout << ek.max_flow(source, sink);

```

Ford-Fulkerson (DFS Augmentation) — $O(E \times \text{maxFlow})$ / $O(V \times E)$ with capacity scaling

```

struct FordFulkerson {
    struct Edge {
        int to, rev;
        long long cap;
    };

    int n;
    vector<vector<Edge>> graph;
    vector<bool> visited;

    FordFulkerson(int n) : n(n), graph(n), visited(n) {}

    void add_edge(int from, int to, long long cap) {
        graph[from].push_back({to, (int)graph[to].size(), cap});
        graph[to].push_back({from, (int)graph[from].size() - 1, 0}); // reverse
        edge cap = 0
    }

    long long dfs(int v, int t, long long f) {
        if (v == t) return f;
        visited[v] = true;
        for (auto& e : graph[v]) {

```

```

        if (!visited[e.to] && e.cap > 0) {
            long long d = dfs(e.to, t, min(f, e.cap));
            if (d > 0) {
                e.cap -= d;
                graph[e.to][e.rev].cap += d;
                return d;
            }
        }
    }
    return 0;
}

long long max_flow(int s, int t) {
    long long flow = 0;
    while (true) {
        fill(visited.begin(), visited.end(), false);
        long long d = dfs(s, t, LLONG_MAX);
        if (d == 0) break; // no augmenting path found
        flow += d;
    }
    return flow;
}

};

// Usage:
// FordFulkerson ff(n);
// ff.add_edge(u, v, cap);
// cout << ff.max_flow(source, sink);

```

min cut

```

struct Dinic {
    struct Edge {
        int to, rev;
        long long cap;
        long long original_cap;
    };

    int n;
    vector<vector<Edge>> graph;
    vector<int> level, iter;

    Dinic(int n) : n(n), graph(n), level(n), iter(n) {}

    void add_edge(int from, int to, long long cap) {
        graph[from].push_back({to, (int)graph[to].size(), cap, cap});
        graph[to].push_back({from, (int)graph[from].size() - 1, 0, 0});
    }

    bool bfs(int s, int t) {

```

```

    fill(level.begin(), level.end(), -1);
    queue<int> q;
    level[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto& e : graph[v]) {
            if (e.cap > 0 && level[e.to] < 0) {
                level[e.to] = level[v] + 1;
                q.push(e.to);
            }
        }
    }
    return level[t] >= 0;
}

long long dfs(int v, int t, long long f) {
    if (v == t) return f;
    for (int& i = iter[v]; i < (int)graph[v].size(); i++) {
        Edge& e = graph[v][i];
        if (e.cap > 0 && level[v] < level[e.to]) {
            long long d = dfs(e.to, t, min(f, e.cap));
            if (d > 0) {
                e.cap -= d;
                graph[e.to][e.rev].cap += d;
                return d;
            }
        }
    }
    return 0;
}

long long max_flow(int s, int t) {
    long long flow = 0;
    while (bfs(s, t)) {
        fill(iter.begin(), iter.end(), 0);
        long long d;
        while ((d = dfs(s, t, LLONG_MAX)) > 0)
            flow += d;
    }
    return flow;
}

// returns nodes in S side (reachable from source in residual graph)
vector<bool> min_cut_side(int s) {
    vector<bool> visited(n, false);
    queue<int> q;
    q.push(s);
    visited[s] = true;
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (auto& e : graph[v]) {
            if (e.cap > 0 && !visited[e.to]) {
                visited[e.to] = true;
            }
        }
    }
}

```

```

        q.push(e.to);
    }
}
}
return visited; // visited[i] = true means node i is in S side
}

// returns min cut edges {from, to}
vector<pair<int,int>> min_cut_edges(int s) {
    vector<bool> visited = min_cut_side(s);
    vector<pair<int,int>> cut;
    for (int v = 0; v < n; v++) {
        if (visited[v]) { // v in S
            for (auto& e : graph[v]) {
                if (!visited[e.to] // e.to in T
                    && e.original_cap > 0 // real forward edge
                    && e.cap == 0) { // fully saturated
                    cut.push_back({v, e.to});
                }
            }
        }
    }
    return cut;
}

bool dfs_find_path(int v, int t, vector<int>& path) {
    path.push_back(v);
    if (v == t) return true;

    for (auto &e : graph[v]) {
        long long flow = e.original_cap - e.cap;
        if (flow > 0) {
            // use this edge
            e.cap += 1; // reduce flow by 1
            graph[e.to][e.rev].cap -= 1;

            if (dfs_find_path(e.to, t, path))
                return true;

            // backtrack
            e.cap -= 1;
            graph[e.to][e.rev].cap += 1;
        }
    }

    path.pop_back();
    return false;
}

vector<vector<int>> get_paths(int s, int t) {
    vector<vector<int>> paths;

    while (true) {
        vector<int> path;

```

```

        if (!dfs_find_path(s, t, path)) break;
        paths.push_back(path);
    }

    return paths;
}

// returns min cut value (should equal max flow)
long long min_cut_value(int s) {
    auto cut = min_cut_edges(s);
    long long val = 0;
    for (auto& [u, v] : cut)
        val += /* find original cap */ 1; // if unit graph
    return val;
}

// cut for König's theorem
vector<pair<int,int>> cut (int s , int t , int nn){
    vp res;
    queue<int>q; q.push(s);
    vi vis(n) ; vis[s] = 1;
    while (!q.empty())
    {
        int u = q.front(); q.pop();
        for(auto [to , rev , cap , org] : graph[u]){
            if(!vis[to] && cap){
                vis[to] = 1 ;
                q.push(to);
            }
        }
    }
    for(int i = 0; i < nn; i++)if(!vis[i])res.push_back({1, i+1});

    for(int j = 0; j < nn; j++)if(vis[nn + j])res.push_back({2, j+1});
    return res ;
}

};

```

MCMF $O(F \cdot E \log V)$:

```

const int INF = 1e9;

struct Edge {
    int to, rev, cap, cost;
};

struct MCMF {

```

```

int n;
vector<vector<Edge>> g;
vector<int> dist, par_v, par_e, pot;

MCMF(int n) : n(n), g(n), dist(n), par_v(n), par_e(n), pot(n) {}

void addEdge(int u, int v, int cap, int cost) {
    Edge a = {v, (int)g[v].size(), cap, cost};
    Edge b = {u, (int)g[u].size(), 0, -cost};
    g[u].push_back(a);
    g[v].push_back(b);
}

pair<int,int> minCostMaxFlow(int s, int t, int need = INF) {
    int flow = 0, cost = 0;

    while (flow < need) {
        fill(dist.begin(), dist.end(), INF);

        priority_queue<pair<int,int> , vector<pair<int,int>> , greater<>> pq;

        dist[s] = 0;
        pq.push({0, s});

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();

            if (d != dist[u]) continue;

            for (int i = 0; i < g[u].size(); i++) {
                Edge &e = g[u][i];
                if (!e.cap) continue;

                int nd = d + e.cost + pot[u] - pot[e.to];
                if (nd < dist[e.to]) {
                    dist[e.to] = nd;
                    par_v[e.to] = u;
                    par_e[e.to] = i;
                    pq.push({nd, e.to});
                }
            }
        }

        if (dist[t] == INF) break;

        for (int i = 0; i < n; i++)
            if (dist[i] < INF)
                pot[i] += dist[i];

        int f = need - flow;
        for (int v = t; v != s; v = par_v[v])
            f = min(f, g[par_v[v]][par_e[v]].cap);
    }
}

```

```

        flow += f;
        cost += f * pot[t];

        for (int v = t; v != s; v = par_v[v]) {
            Edge &e = g[par_v[v]][par_e[v]];
            e.cap -= f;
            g[v][e.rev].cap += f;
        }
    }

    return {flow, cost};
}
};

```

SCC Tarjan :

```

int n , m ;
vvi adj(n+1) , res;
vi tin(n+1) , low(n+1);
int timer = 1 ;
vector<bool>instack(n+1);
stack<int>st ;
function<void(int)>dfs = [&](int u){
    tin[u] = low[u] = timer++;
    st.push(u);
    instack[u] = true ;

    for(auto v : adj[u]){
        if(!tin[v]){
            dfs(v);
            low[u] = min(low[u] , low[v]);
        }
        else if(instack[v]){
            low[u] = min(low[u], tin[v]);
        }
    }

    if(low[u] == tin[u]){
        vector<int> scc;
        while (true) {
            int x = st.top();
            st.pop();

            instack[x] = false;
            scc.push_back(x);

            if (x == u)
                break;
        }
    }
}

```



```

        res.push_back(scc);
    }
};
for (int i = 1; i <= n; i++)
{
    if(!tin[i])dfs(i);
}

```

Bridges In Graphs :

```

vvi adj(n+1);
vi tin(n+1) , low(n+1) , vis(n+1);
int timer = 1 , cnt = 0;
function<void(int,int)>dfs = [&](int u , int p){
    tin[u] = low[u] = timer++;
    vis[u] = 1 ;
    bool skp = false;
    for(auto v : adj[u]){
        if(!skp && v == p){
            skp = true ; continue;
        }
        if(vis[v]){
            low[u] = min(low[u] , tin[v]);
        }
        else{
            dfs(v , u);
            low[u] = min(low[u] , low[v]);
            if (low[v] > tin[u]){
                mp[{min(u,v) , max(u,v)}]++;
            }
        }
    }
};
for (int i = 0; i < n; i++)
{
    if(!vis[i])dfs(i , -1);
}

```

2-SAT :

```

/*
===== 2-SAT Cheat Sheet =====
1) x OR y
addOr(x, true, y, true);

```

```

=====

2)  $x \rightarrow y \equiv !x \vee y$ 
addOr(x, false, y, true);

-----

 $y \rightarrow x$ 
addOr(y, false, x, true);
=====

3)  $x \equiv y$ 

Equivalent to :  $(x \rightarrow y) \wedge (y \rightarrow x)$ 
addOr(x, true, y, false);
addOr(x, false, y, true);
=====

4) Inequality (XOR)
 $x \neq y$ 

 $(x \vee y) \wedge (!x \vee !y)$ 

addOr(x, true, y, true);
addOr(x, false, y, false);

=====

5) Force  $x = \text{true}$ 
addOr(x, true, x, true);
=====

6) Force  $x = \text{false}$ 
addOr(x, false, x, false);
=====

7) At least one is true

 $x \vee y$ 
addOr(x, true, y, true);
=====

8) At most one is true

 $!(x \wedge y) = !x \vee !y$ 

addOr(x, false, y, false);
=====

9) Exactly one is true
Equivalent to :  $x \oplus y$ 
addOr(x, true, y, true);
addOr(x, false, y, false);
=====

*/
struct TwoSAT {
    int n;
    vector<vector<int>> adj;

    vector<int> dfn, low, comp, st;
    vector<bool> inStack;

```

```
int timer = 0, sccCnt = 0;

TwoSAT(int N) {
    n = N;
    adj.assign(2 * n, {});
}

int id(int x, bool val) {
    return 2 * x + val;
}

void addOr(int x, bool xv, int y, bool yv) {
    adj[id(x, !xv)].push_back(id(y, yv));
    adj[id(y, !yv)].push_back(id(x, xv));
}

void tarjan(int u){
    dfn[u] = low[u] = ++timer;
    st.push_back(u);
    inStack[u] = true;

    for (int v : adj[u]) {
        if (!dfn[v]) {
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (inStack[v]) {
            low[u] = min(low[u], dfn[v]);
        }
    }

    if (low[u] == dfn[u]) {
        while (true) {
            int v = st.back();
            st.pop_back();
            inStack[v] = false;
            comp[v] = sccCnt;
            if (v == u) break;
        }
        sccCnt++;
    }
}

pair<bool, vector<bool>> solve(){
    int m = 2 * n;

    dfn.assign(m, 0);
    low.assign(m, 0);
    comp.assign(m, -1);
    inStack.assign(m, false);

    timer = 0; sccCnt = 0; st.clear();

    for (int i = 0; i < m; i++){
```

```

        if (!dfn[i]) tarjan(i);
    }

    vector<bool> ans(n);

    for (int i = 0; i < n; i++) {
        if (comp[id(i, false)] == comp[id(i, true)])
            return {false, {}};

        ans[i] = comp[id(i, false)] > comp[id(i, true)];
    }

    return {true, ans};
};

int addAtMostOne(vector<pair<int,int>> vars, TwoSAT &tw, int nextVar) {

    int k = vars.size();
    if (k <= 1) return nextVar;

    vector<int> s(k - 1);

    for (int i = 0; i < k - 1; i++) s[i] = nextVar++;
    // x1 -> s1
    tw.addOr(vars[0].first, !vars[0].second, s[0], true);

    for (int i = 1; i < k - 1; i++) {

        // xi -> si
        tw.addOr(vars[i].first, !vars[i].second, s[i], true);
        // s(i-1) -> si
        tw.addOr(s[i - 1], false, s[i], true);
        // xi -> !s(i-1)
        tw.addOr(vars[i].first, !vars[i].second, s[i - 1], false);
    }
    // xn -> !s_last
    tw.addOr(vars[k - 1].first, !vars[k - 1].second, s[k - 2], false);
    return nextVar;
}

```

Centroid:

```

struct centroid
{
    int n , ans = 0 , k;
    vector<vector<int>> adj;
    vector<int> par , dead , siz , dis , cnt;

```

```

centroid(int _n , vector<vector<int>> & _adj , int _k)
:n(_n), k(_k) ,adj(_adj), par(n+1), siz(n+1), dead(n+1) , dis(n+1) , cnt(n+1)
{}

void calcSize(int u, int p) {
    siz[u] = 1;
    for (int v : adj[u]) {
        if (v == p || dead[v]) continue;
        calcSize(v, u);
        siz[u] += siz[v];
    }
}

int findC(int u, int p, int tot) {
    for (int v : adj[u]) {
        if (v == p || dead[v]) continue;
        if (siz[v] > tot / 2)return findC(v, u, tot);
    }
    return u;
}

void calcDis(int u, int p , int d , vi&cur) {
    if(d > k) return ;
    cur.push_back(d);
    for (int v : adj[u]) {
        if (v == p || dead[v]) continue;
        calcDis(v, u , d+1 , cur);
    }
}

void calc(int u){
    int mx = 0 ;
    cnt[0] = 1 ;
    for(auto v : adj[u]){

        if(dead[v]) continue;

        vi curDis;
        calcDis(v , u , 1 , curDis);

        for(auto d : curDis){
            ans += cnt[k-d];
        }
        for(auto d : curDis){
            cnt[d]++; mx = max(d,mx);
        }
    }

    for(int i = 0 ; i <= mx ; i++){
        cnt[i] = 0 ;
    }
}

void build(int u = 1, int p = -1, int d = 0) {
    calcSize(u, -1);
}

```

```

    int c = findC(u, -1, siz[u]);
    calc(c);
    par[c] = p;
    dis[c] = d;
    dead[c] = 1;

    for(auto v : adj[c]){
        if(!dead[v]) build(v, c, d+1);
    }
}
};

```

Centroid Update and Query:

```

struct centroid
{
    int n;
    vector<vector<int>>>adj;
    vector<vector<pair<int,int>>>>path;
    vector<int> par, dead, siz, dis, cnt, best;

    centroid(int _n, vector<vector<int>>> & _adj)
        :n(_n), adj(_adj), best(n+1, 1e18), par(n+1), siz(n+1), dead(n+1), dis(n+1), cnt(n+1), path(n+1){}

    void calcSize(int u, int p) {
        siz[u] = 1;
        for (int v : adj[u]) {
            if (v == p || dead[v]) continue;
            calcSize(v, u);
            siz[u] += siz[v];
        }
    }

    int findC(int u, int p, int tot) {
        for (int v : adj[u]) {
            if (v == p || dead[v]) continue;
            if (siz[v] > tot / 2) return findC(v, u, tot);
        }
        return u;
    }

    void calcDis(int u, int p, int d, int c) {
        path[u].push_back({d,c});
        for (int v : adj[u]) {
            if (v == p || dead[v]) continue;
            calcDis(v, u, d+1, c);
        }
    }
}

```

```

}

void build(int u = 1, int p = -1, int d = 0) {
    calcSize(u, -1);
    int c = findC(u, -1, siz[u]);
    par[c] = p;
    dis[c] = d;
    dead[c] = 1;
    calcDis(c, -1, 0, c);

    for(auto v : adj[c]){
        if(!dead[v])build(v, c, d+1);
    }
}

void update(int u){
    for(auto [d, c] : path[u]){
        best[c] = min(best[c], d);
    }
}

int query(int u){
    int ans = 1e18;
    for(auto [d, c] : path[u]){
        ans = min(ans, best[c] + d);
    }
    return ans;
}

};

```

Centroid Collect and Remove:

```

const ll N = 2e5 + 10, M = (1LL << 20);
vector<ll> adj[N];
ll removed[N], frq[M], sz[N], a[N], res[N];
ll dfsSZ(ll u, ll p) {
    sz[u] = 1;
    for (ll v: adj[u])
        if (v != p and !removed[v])
            sz[u] += dfsSZ(v, u);
    return sz[u];
}

ll center(ll u, ll p, ll tot) {
    for (ll v: adj[u])
        if (v != p and !removed[v] and sz[v] * 2 > tot)
            return center(v, u, tot);
    return u;
}

```

```

ll dfs(ll u, ll p, ll cur) {
    cur ^= a[u];
    ll tmp = frq[cur];
    for (ll bit = 0; bit < 20; bit++)
        tmp += frq[cur ^ (1LL << bit)];
    for (ll v: adj[u])
        if (v != p and !removed[v])
            tmp += dfs(v, u, cur);
    res[u] += tmp;
    return tmp;
}

void collect(ll u, ll p, ll add, ll cur) {
    cur ^= a[u];
    frq[cur] += add;
    for (ll v: adj[u])
        if (!removed[v] and v != p)
            collect(v, u, add, cur);
}

void decompose(ll u) {
    u = center(u, u, dfsSZ(u, u));
    removed[u] = 1;
    collect(u, -1, 1, 0);
    ll tmp = frq[0] + 1;
    for (ll bit = 0; bit < 20; bit++)
        tmp += frq[(1LL << bit)];
    for (ll v: adj[u]) {
        if (removed[v]) continue;
        collect(v, -1, -1, a[u]);
        tmp += dfs(v, u, 0);
        collect(v, -1, 1, a[u]);
    }
    collect(u, -1, -1, 0);
    res[u] += tmp / 2;
    for (ll v: adj[u])
        if (!removed[v])
            decompose(v);
}

```

Hopcraft

```

struct HopcroftKarp {
    int n, m; // n: nodes in left set, m: nodes in right set
    vector<vector<int>> adj;
    vector<int> match_left, match_right, dist;
    const int INF = 1e9;

    HopcroftKarp(int n, int m) : n(n), m(m), adj(n),

```



```

        match_left(n, -1), match_right(m, -1), dist(n) {}

void add_edge(int u, int v) {
    adj[u].push_back(v);
}

// BFS to find the shortest augmenting paths
bool bfs() {
    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (match_left[i] == -1) {
            dist[i] = 0;
            q.push(i);
        } else {
            dist[i] = INF;
        }
    }

    bool found = false;
    int min_dist = INF;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        if (dist[u] < min_dist) {
            for (int v : adj[u]) {
                if (match_right[v] == -1) {
                    min_dist = dist[u] + 1;
                    found = true;
                } else if (dist[match_right[v]] == INF) {
                    dist[match_right[v]] = dist[u] + 1;
                    q.push(match_right[v]);
                }
            }
        }
    }

    return found;
}

// DFS to augment the matching based on BFS distance levels
bool dfs(int u) {
    for (int v : adj[u]) {
        if (match_right[v] == -1) {
            match_left[u] = v;
            match_right[v] = u;
            return true;
        } else if (dist[match_right[v]] == dist[u] + 1) {
            if (dfs(match_right[v])) {
                match_left[u] = v;
                match_right[v] = u;
                return true;
            }
        }
    }
}

```

```

    }
    dist[u] = INF; // Avoid re-scanning nodes that don't lead to a path
    return false;
}

int max_matching() {
    int matching = 0;
    while (bfs()) {
        for (int i = 0; i < n; i++) {
            if (match_left[i] == -1 && dfs(i)) {
                matching++;
            }
        }
    }
    return matching;
}

vector<pair<int, int>> get_matching_pairs() {
    vector<pair<int, int>> pairs;
    for (int u = 0; u < n; u++) {
        if (match_left[u] != -1) {
            pairs.push_back({u, match_left[u]});
        }
    }
    return pairs;
}
};

vll graph;
vll color, leftId, rightId;
int L = 0, R = 0;

void dfs(int u, int p, int c) {
    color[u] = c;

    if (c == 0) leftId[u] = L++;
    else rightId[u] = R++;

    for (int v : graph[u]) {
        if (v == p) continue;
        dfs(v, u, c ^ 1);
    }
}

void solve() {
    ll n; cin >> n;
    graph.assign(n+1, vll(0));
    color.assign(n + 1, -1);
    leftId.assign(n + 1, -1);
    rightId.assign(n + 1, -1);
    L = R = 0;

    for (ll i = 0; i < n-1; i++) {
        ll u, v;

```

```
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    color[1] = 1;
    dfs(1, -1, 0);
    auto ans = HopcroftKarp(L, R);
    for (int u = 1; u <= n; u++) {
        if (color[u] == 0) {
            for (int v : graph[u]) {
                ans.add_edge(leftId[u], rightId[v]);
            }
        }
    }

    cout << ans.max_matching() << endl;
}
```